第二讲 课后作业：黄旭华传记序言的词频统计

> **作业要求（PPT 第 40 页「六、课后作业与提升」）**
>
> 1. 完成 ppt 中的程序运行，包括**全文词频统计**、**指定类型词频统计**；
> 2. 链接小组案例：把 ppt 中的文本换成功勋科学家**黄旭华**院士的传记序言文本 （`科学家博物馆-黄旭华传记序言.txt`），1）统计**全文词频**；2）统计**指定词频**，如"黄旭华"；
> 3. 兴趣提升：阅读本讲提到的学术论文。

本 notebook 对应要求 1 和 2，并把 PPT 里提到的两个拓展（TF-IDF、齐普夫定律）各做了一遍验证。 所有数据都取自本文件夹，不依赖任何外部下载。

0 环境准备

分词用 `jieba`，画图用 `matplotlib`，表格用 `pandas`，词云用 `wordcloud`。 如果本机还没装，取消下面这行的注释再运行：

In [1]:
```
# !pip install jieba wordcloud -i https://pypi.tuna.tsinghua.edu.cn/simple
```

In [2]:
```
import jieba
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter
from matplotlib import font_manager
from wordcloud import WordCloud

# 中文字体：课程示例用的是 mac 的 'Heiti TC'，Windows 上换成微软雅黑
font_manager.fontManager.addfont('C:/Windows/Fonts/msyh.ttc')
plt.rcParams['font.sans-serif'] = ['Microsoft YaHei']
plt.rcParams['axes.unicode_minus'] = False

FONT_PATH = 'C:/Windows/Fonts/msyh.ttc'
print('环境就绪，jieba 版本：', jieba.__version__)
```

```
环境就绪，jieba 版本： 0.42.1
```

In [3]:
```
# 作业要用的文件都在不在
import os

for f in ['科学家博物馆-黄旭华传记序言.txt', 'sanguo.txt', '人民日报-杭州亚运会-科技智能.txt', '知了.txt']:
    size = os.path.getsize(f) if os.path.exists(f) else 0
    print(f'{f:<38s} {"存在" if size else "缺失"}  {size} 字节')
```

```
科学家博物馆-黄旭华传记序言.txt                    存在  11896 字节
sanguo.txt                                  存在  1240068 字节
人民日报-杭州亚运会-科技智能.txt                   存在  6142 字节
知了.txt                                    存在  3124 字节
```

1 全文词频统计

1.1 读取文本

> **坑 1：编码。** 中文 txt 常见 `utf-8` 和 `GB18030`（Windows 记事本默认）两种编码，用错了就是乱码或 `UnicodeDecodeError`——文件夹里那张 `图片乱码的代码修正.PNG` 讲的就是这件事。
>
> **坑 2：换行符。** Windows 文本的行尾是 `\r\n`，jieba 会把它当成一个「词」。如果只把 `\n` 放进停用词表，`\r\n` 会漏网，然后霸占词频榜（本文件实测它会排进前 6）。读进来就统一成 `\n` 最省事。

In [4]:
```python
path = '科学家博物馆-黄旭华传记序言.txt'

raw = open(path, 'rb').read()
for enc in ('utf-8', 'gb18030', 'utf-8-sig'):
    try:
        article = raw.decode(enc)
        print(f'用 {enc} 解码成功，共 {len(article)} 个字符')
        break
    except UnicodeDecodeError:
        print(f'用 {enc} 解码失败，换一个编码再试')

# 统一换行符，避免 \r\n 被当成一个词
article = article.replace('\r\n', '\n').replace('\r', '\n')

print('\n--- 开头 200 字 ---')
print(article[:200])
```

```
用 utf-8 解码成功，共 4070 个字符

--- 开头 200 字 ---
在核潜艇领域，我国已形成一套完整的研究、设计、试验、制造、测试的核潜艇产业体系，而且装备了一支具有极高战略威慑力的、成梯次配备的、已近实现战备巡逻的核潜艇部队。回顾我国核潜艇的发展历程，人们自然会想起以黄旭华为代表的五位两院院士及无数第一代核潜艇研制人员的皓首穷经、筚路蓝缕、无私奉献，正是他们所铸就的国之重器使我国彻底摆脱了超级大国的核讹诈，更使我们在民族复兴的道路上迈出了坚实的一步。

而今，由
```

1.2 分词，并补充词典里没有的新词

课程代码里有一行 `jieba.add_word('国立交通大学')`：jieba 的默认词典会把「国立交通大学」切成「国立 / 交通大学」，加进词典后才会整体切出。这是**自定义词典**最直接的用法。

In [5]:
```
jieba.add_word('国立交通大学')   # 加入字典中没有的新词

words = list(jieba.cut(article))
print('分出来的词数（含标点）：', len(words))
print('前 40 个词：')
print('/'.join(words[:40]))
```

```
Building prefix dict from the default dictionary ...
Loading model from cache C:\Users\34669\AppData\Local\Temp\jieba.cache
Loading model cost 0.316 seconds.
Prefix dict has been built successfully.
```

```
分出来的词数（含标点）：  2290
前 40 个词：
在/核潜艇/领域/，/我国/已/形成/一套/完整/的/研究/、/设计/、/试验/、/制造/、/测试/的/核潜艇/产业/体系/，/而且/装备/了/一支/具有/极高/战略/威慑力/的/、/成/梯次/配备/的/、/已近
```

1.3 去掉标点、符号和单字

为什么必须过滤？ 不过滤的话，`，`、`。`、`的` 会霸占前几名，把真正的主题词挤下去。
课程里用一个集合 `dele` 手工列出要剔除的符号——这是最朴素、也最好理解的停用词表。

In [6]:
```
dele = {'。', '！', '？', '的', '“', '”', '（', '）', '，', '》', '《', '，', '、',
        '\n', '\t', '；', '：', '·', '—', '-', '%', '…', '‘', '’', '(', ')', '《',
'》'}

def is_meaningful(word):
    """去掉空白词、停用符号，以及长度为 1 的字。"""
    return word.strip() != '' and word not in dele and len(word) > 1

kept = [w for w in words if is_meaningful(w)]
print('过滤前：', len(words), '个词')
print('过滤后：', len(kept), '个词    （剔除了', len(words) - len(kept), '个标点/单字）')
```

```
过滤前：  2290 个词
过滤后：  1390 个词    （剔除了 900 个标点/单字）
```

In [7]:
```
# 直观看看被过滤掉的都是什么
removed = [w for w in words if not is_meaningful(w)]
print('被过滤掉最多的 15 个「词」：')
for w, n in Counter(removed).most_common(15):
    print(f'  {repr(w):<8s} {n} 次')
```

```
被过滤掉最多的 15 个「词」：
  '的'      169 次
  ', '      135 次
  '、'      101 次
  '。'      61 次
  '了'      42 次
  '\n'     40 次
  '在'      23 次
  '与'      22 次
  '对'      19 次
  '其'      17 次
  '“'      14 次
  '”'      13 次
  '及'      12 次
  '是'      12 次
  '所'      11 次
```

1.4 统计词频：字典 → 排序 → 表格

课程代码用「字典 + `list.count()`」，逻辑直观但慢（每个词都要把全文重扫一遍）。这里保留课程写法，同时给出结果一致、但快得多的 `Counter` 写法。

In [8]:
```python
# —— 课程写法：字典 + count ——
articleSet = set(kept) - dele          # 集合去重
articleDict = {}                        # 词 → 词频
for w in articleSet:
    articleDict[w] = kept.count(w)      # 字典的 key-value 配对

# —— 等价写法：Counter ——
counter = Counter(kept)

print('两种写法结果一致吗？', dict(counter) == articleDict)
print('词表大小（去重后）:', len(articleDict))
print('词次总数:', sum(articleDict.values()))
```

```
两种写法结果一致吗？ True
词表大小（去重后）: 723
词次总数: 1390
```

In [9]:
```python
articlelist = sorted(articleDict.items(), key=lambda x: x[1], reverse=True)  # 按词频降序

df_top = pd.DataFrame(articlelist[:30], columns=['词', '词频'])
df_top.index = df_top.index + 1        # 名次从 1 开始
df_top.index.name = '名次'
df_top
```

Out[9]:

名次	词	词频
1	黄旭华	53
2	核潜艇	32
3	采集	29
4	学术	22
5	资料	21
6	工作	17
7	成长	15
8	小组	14
9	院士	13
10	专业	13
11	进行	13
12	我国	12
13	技术	12
14	研制	12
15	工程	11
16	访谈	10
17	第一代	8
18	科学	8
19	介绍	8
20	主要	8
21	传记	7
22	人生	7
23	及其	7
24	历史	7
25	思想	7
26	成就	6
27	要求	6
28	一生	6
29	传主	6
30	过程	6

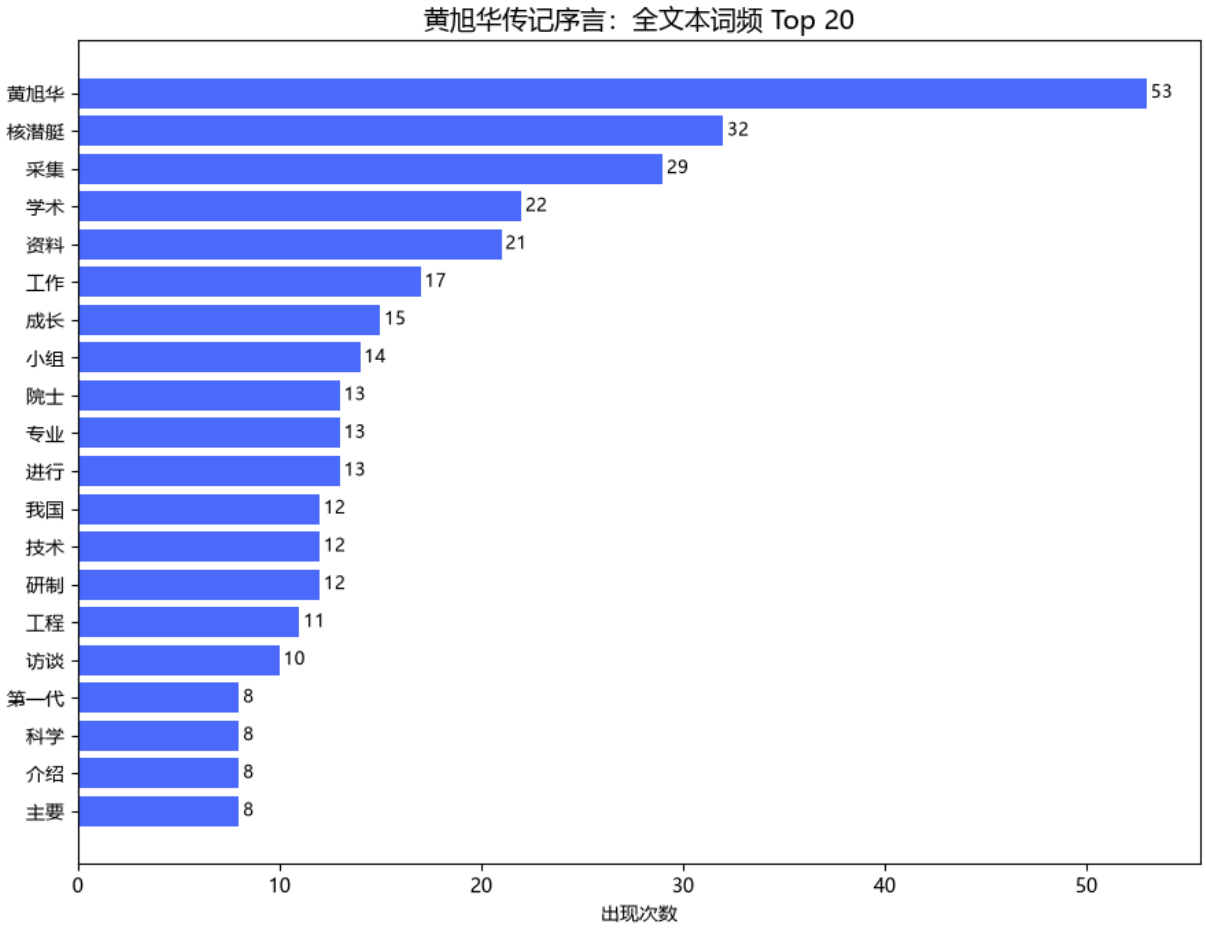
In [10]:
```
# 也可以按课程代码那样直接 print 前 20 个
for i in range(20):
    print(articlelist[i])
```

```
('黄旭华', 53)
('核潜艇', 32)
('采集', 29)
('学术', 22)
('资料', 21)
('工作', 17)
('成长', 15)
('小组', 14)
('院士', 13)
('专业', 13)
('进行', 13)
('我国', 12)
('技术', 12)
('研制', 12)
('工程', 11)
('访谈', 10)
('第一代', 8)
('科学', 8)
('介绍', 8)
('主要', 8)
```

In [11]:

```python
# Top 20 柱状图
top20 = articlelist[:20]
labels = [w for w, _ in top20][::-1]
values = [n for _, n in top20][::-1]

plt.figure(figsize=(9, 7))
bars = plt.barh(labels, values, color='#4d6bfe')
plt.xlabel('出现次数')
plt.title('黄旭华传记序言：全文本词频 Top 20', fontsize=14)
for bar, v in zip(bars, values):
    plt.text(v + 0.2, bar.get_y() + bar.get_height() / 2, str(v), va='center',
fontsize=9)
plt.tight_layout()
plt.show()
```



1.5 词云图

PPT 里用的是在线工具「微词云」，本地几行代码就能做出同样的效果。

In [12]:
```
wc = WordCloud(
    font_path=FONT_PATH,
    width=1200, height=700,
    background_color='white',
    max_words=120,
).generate_from_frequencies(dict(articlelist))

plt.figure(figsize=(12, 7))
plt.imshow(wc, interpolation='bilinear')
plt.axis('off')
plt.title('黄旭华传记序言：词云', fontsize=14)
plt.tight_layout()
plt.show()

wc.to_file('homework_wordcloud.png')
print('词云已保存为 homework_wordcloud.png')
```



词云已保存为 `homework_wordcloud.png`

2 指定类型词频统计

「指定词频」和「全文词频」的区别是：**我们已经知道要看哪些词**，只统计它们出现了多少次。 课程代码用的是 `data_txt.count(term)` ——本质是**字符串子串匹配**，不经过分词。

2.1 定义要统计的词

In [13]:
```
terms = ['黄旭华', '核潜艇', '国立交通大学', '院士', '第一代', '核动力', '中国工程院']
print('本次要统计的指定词：', terms)
```

本次要统计的指定词： ['黄旭华', '核潜艇', '国立交通大学', '院士', '第一代', '核动力', '中国工程院']

2.2 用 `str.count()` 统计（课程的写法）

In [14]:
```
terms_dict = {}
for term in terms:
    terms_dict[term] = article.count(term)   # 字符串匹配，不经过分词

df_terms = pd.DataFrame(list(terms_dict.items()), columns=['指定词', '出现次数'])
df_terms
```

Out[14]:

	指定词	出现次数
0	黄旭华	59
1	核潜艇	32
2	国立交通大学	3
3	院士	13
4	第一代	8
5	核动力	1
6	中国工程院	1

2.3 两种口径的差异：`str.count()` vs `jieba` 计数

`str.count('核潜艇')` 是把「核潜艇」当**字符串**在全文里找；分词后计数只统计「**独立成词**」的次数。两者的差值暴露了分词的真实问题——而且这次问题恰好出在**人名**上。

In [15]:

```
rows = []
for term in terms:
    by_count = article.count(term)                    # 子串匹配
    by_jieba = kept.count(term)                       # 分词后独立成词的次数
    rows.append({'指定词': term, 'str.count': by_count, 'jieba分词计数': by_jieba,
                 '差值': by_count - by_jieba})

df_compare = pd.DataFrame(rows)
df_compare
```

Out[15]:

	指定词	str.count	jieba分词计数	差值
0	黄旭华	59	53	6
1	核潜艇	32	32	0
2	国立交通大学	3	3	0
3	院士	13	13	0
4	第一代	8	8	0
5	核动力	1	1	0
6	中国工程院	1	1	0

「黄旭华」差了 6 次，为什么？把分词结果摊开看就明白了：

In [16]:

```
# 所有「含黄旭华」的分词结果，按出现次数排
from collections import Counter

name_tokens = Counter(t for t in words if '黄旭华' in t)
print('分词里包含「黄旭华」的 token：')
for tok, n in name_tokens.most_common():
    print(f'  {tok!r}  x{n}')

print('\n注意：还有 4 处「黄旭华」被切成了别的样子，用 tokenize 的偏移量才能定位到。')
```

```
分词里包含「黄旭华」的 token：
  '黄旭华'  x53
  '黄旭华出'  x2
```

注意：还有 4 处「黄旭华」被切成了别的样子，用 tokenize 的偏移量才能定位到。

In [17]:
```python
# 用 tokenize 拿到每个 token 的字符区间，找出「没被任何 token 完整覆盖」的人名
covered = set()
for w, s, e in jieba.tokenize(article):
    if '黄旭华' in w:
        covered.update(range(s, e))

import re
missed = [m.start() for m in re.finditer('黄旭华', article) if m.start() not in
covered]
print(f'被切碎、没被完整算进去的人名：{len(missed)} 处\n')
for pos in missed:
    ctx = article[max(0, pos - 4): pos + 6]
    toks = [t for t, s, e in jieba.tokenize(ctx)]
    print(f'  原文：…{ctx}…')
    print(f'    切成：{toks}')
```

```
被切碎、没被完整算进去的人名：4 处

  原文：…会想起以黄旭华为代表…
    切成：['会', '想起', '以', '黄旭', '华为', '代表']
  原文：…命斗争是黄旭华人生的…
    切成：['命', '斗争', '是', '黄旭', '华人', '生', '的']
  原文：…程不仅是黄旭华人生的…
    切成：['程', '不仅', '是', '黄旭', '华人', '生', '的']
  原文：…重点对以黄旭华为代表…
    切成：['重点', '对以', '黄旭', '华为', '代表']
```

为什么会切碎？——「华为」和「华人」抢走了字

jieba 的词典里有 `华为`、`华人` 这两个高频词，于是：

原文	jieba 的切分	问题
以**黄旭华为**代表	`以` / `黄旭` / `华为` / `代表`	「华为」把「华 + 为」抢走了
黄旭华人生的第一个高潮	`黄旭` / `华人` / `生` / `的`	「华人」把「华 + 人」抢走了
是**黄旭华人生**的重要转折点	`黄旭` / `华人` / `生` / `的`	同上

这两处一共吃掉 4 次出现，加上前面 2 次被粘成 `黄旭华出`（`黄旭华出生`），分词计数就从 59 掉到了 53。**这就是「分词会改变统计结果」最直观的证据。**

修法：把人物姓名加进词典

课程里 `jieba.add_word('国立交通大学')` 就是这个道理。给「黄旭华」加词，并给一个较高的词频，让它在与「华为」「华人」竞争时胜出：

In [18]:
```python
jieba.add_word('黄旭华', freq=100000)   # freq 要足够高，才能压过「华为」「华人」

fixed = list(jieba.cut(article))
```

```
print('加词后「黄旭华」的分词计数：', fixed.count('黄旭华'))
print('原文 str.count     ：', article.count('黄旭华'))
print('两者是否一致：', fixed.count('黄旭华') == article.count('黄旭华'))

print('\n再看之前被切碎的那句：')
print([t for t, s, e in jieba.tokenize('以黄旭华为代表的五位两院院士')])
```

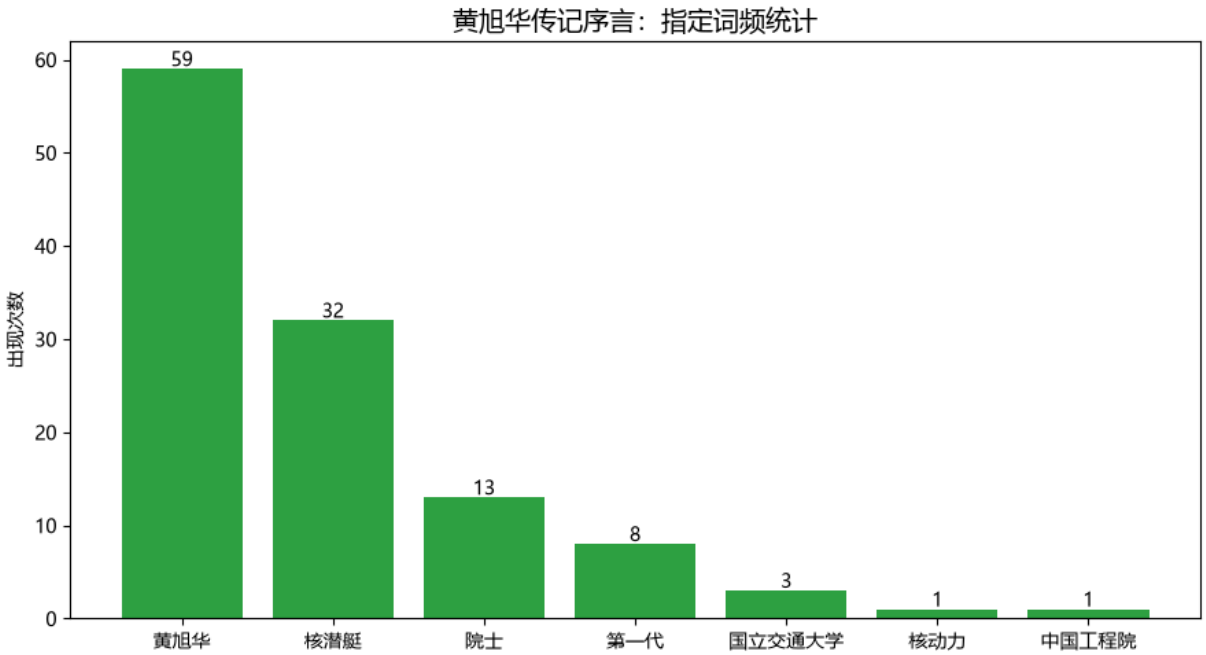
```
加词后「黄旭华」的分词计数：  59
原文 str.count     ：  59
两者是否一致：  True

再看之前被切碎的那句：
['以', '黄旭华', '为', '代表', '的', '五位', '两院', '院士']
```

2.4 指定词频柱状图

In [19]:
```
df_plot = df_terms.sort_values('出现次数', ascending=False)

plt.figure(figsize=(9, 5))
bars = plt.bar(df_plot['指定词'], df_plot['出现次数'], color='#2ea043')
plt.ylabel('出现次数')
plt.title('黄旭华传记序言：指定词频统计', fontsize=14)
for bar, v in zip(bars, df_plot['出现次数']):
    plt.text(bar.get_x() + bar.get_width() / 2, v + 0.3, str(v), ha='center',
fontsize=10)
plt.tight_layout()
plt.show()
```



2.5 词频之外：看看「黄旭华」都出现在什么语境里

这是词频统计的下一步——**上下文（concordance）**。同样的 59 次，出现在「黄旭华是我国著名船舶专家」和「由黄旭华院士等人所开创」，信息量完全不同。

In [20]:
```
hits = [m.start() for m in re.finditer('黄旭华', article)]
print(f'「黄旭华」共出现 {len(hits)} 次，前 10 处上下文：')
for i, pos in enumerate(hits[:10], 1):
    left = article[max(0, pos - 12):pos].replace('\n', ' ')
    right = article[pos + 3: pos + 17].replace('\n', ' ')
    print(f'  {i:2d}. …{left}【黄旭华】{right}…')
```

```
「黄旭华」共出现 59 次，前 10 处上下文：
 1. …展历程，人们自然会想起以【黄旭华】为代表的五位两院院士及无数第…
 2. …坚实的一步。　而今，由【黄旭华】院士等人所开创的核潜艇工程以…
 3. …中国梦”的伟大重任。　【黄旭华】是我国著名船舶专家、核潜艇研…
 4. …。1924年2月24日，【黄旭华】出生于广东省汕尾市海丰县田墘…
 5. …究设计工作。1958年，【黄旭华】开始参与并领导我国第一代核潜…
 6. …副所长、所长、党委书记。【黄旭华】先后于1978年获全国科学大…
 7. …学技术进步奖特等奖。　【黄旭华】出生于以医为主、兼理农商之家…
 8. …，经过党校系统培训学习，【黄旭华】在政治思想上逐步成熟。经过苏…
 9. …事舰船的转让仿制的锤炼，【黄旭华】在专业技术上也崭露头角。19…
10. …也崭露头角。1958年，【黄旭华】因为政治素质过硬、专业技术精…
```

3 拓展验证：PPT 里提到的两个「更高级的词频统计」

3.1 齐普夫定律（Zipf's Law）

PPT 第 28 页给出 `f·r = C`。两边取对数：`log f = log C − log r`，所以「词频–名次」的双对数图上应该是一条**斜率为负的直线**，严格形式下斜率 ≈ −1。

In [21]:
```
import numpy as np

freqs = [n for _, n in articlelist]
ranks = np.arange(1, len(freqs) + 1)

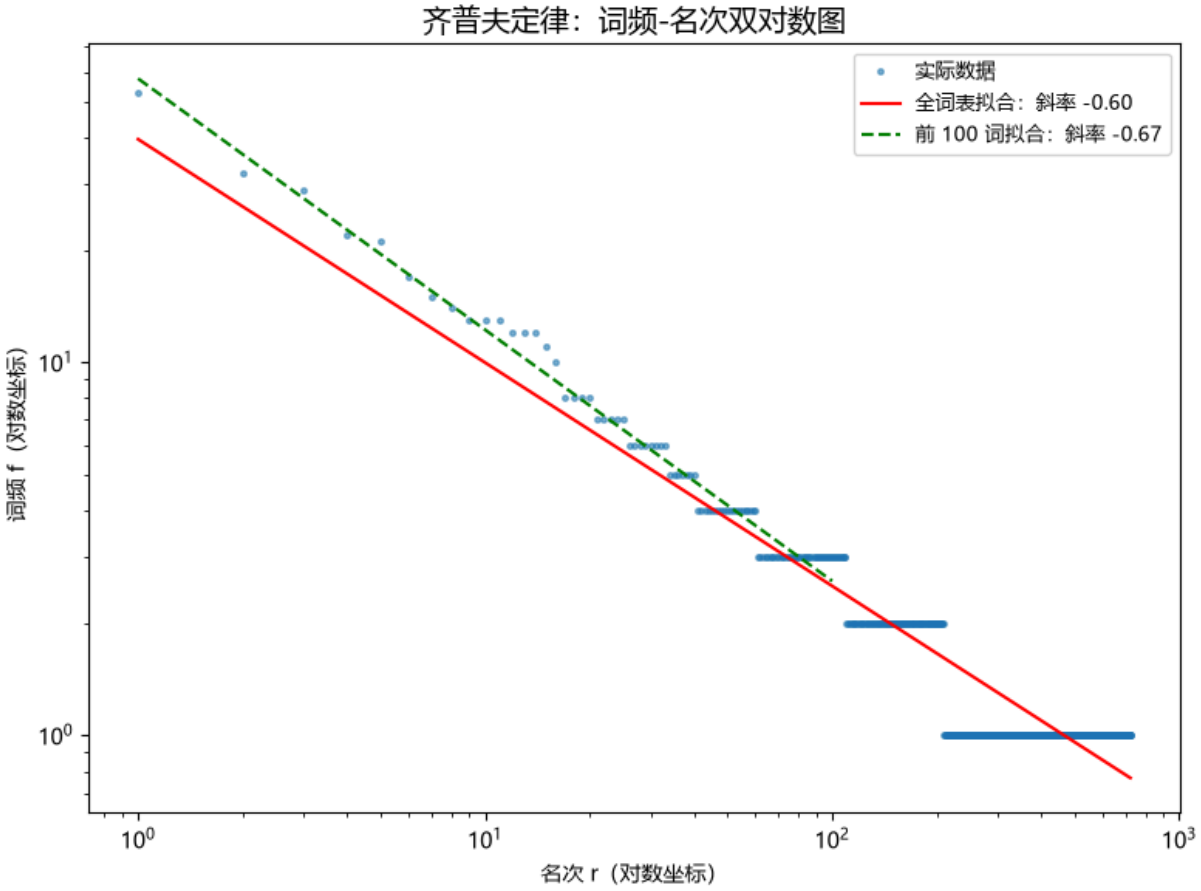
def fit_slope(rank_arr, freq_arr, start=0, end=None):
    end = end or len(rank_arr)
    x, y = np.log10(rank_arr[start:end]), np.log10(freq_arr[start:end])
    k, b = np.polyfit(x, y, 1)
    return k, b

k_all, b_all = fit_slope(ranks, freqs)
k_top, b_top = fit_slope(ranks, freqs, 0, 100)

plt.figure(figsize=(8, 6))
plt.loglog(ranks, freqs, 'o', markersize=2.5, alpha=0.55, label='实际数据')
plt.plot(ranks, 10 ** (k_all * np.log10(ranks) + b_all), 'r-', linewidth=1.5,
         label=f'全词表拟合：斜率 {k_all:.2f}')
plt.plot(ranks[:100], 10 ** (k_top * np.log10(ranks[:100]) + b_top), 'g--',
linewidth=1.5,
         label=f'前 100 词拟合：斜率 {k_top:.2f}')
plt.xlabel('名次 r（对数坐标）')
plt.ylabel('词频 f（对数坐标）')
plt.title('齐普夫定律：词频-名次双对数图', fontsize=14)
plt.legend(fontsize=9)
plt.tight_layout()
```

```
plt.show()

print(f'全词表拟合斜率 = {k_all:.3f}')
print(f'前 100 词拟合斜率 = {k_top:.3f}')
```




```
全词表拟合斜率 = -0.599
前 100 词拟合斜率 = -0.673
```

怎么读这张图？ 点子基本落在一条向下的直线上——**符合齐普夫定律「高频词少、低频词多」的长尾形状。** 但拟合出来的斜率是 **−0.6 ~ −0.7，并不等于 −1**，原因是：

1. **文本太短**：全文只有 4000 字、700 多个不同的词。齐普夫定律是对**大规模语料**的统计规律， 小样本里高频端被「截顶」（`黄旭华` 53 次、`核潜艇` 32 次，头部很平），斜率自然被拉平。
2. **中文分词的影响**：英文按空格切词，中文靠分词器，长词被合并、专名被切开， 词频分布和英文语料不完全可比。
3. **同一篇文本内部**：齐普夫描述的是「一个语料库」的词频分布，单篇短文的波动更大。

所以在小文本上验证时，**该看的是「形状对不对、斜率为不为负」，而不是硬套 −1。** 想得到接近 −1 的斜率，需要换成 `sanguo.txt` 那样几十万字的语料。

3.2 TF-IDF：次数多 ≠ 重要

PPT 第 22 页说得很直白：**「文本中出现次数越多的词，并不一定最重要！」** TF-IDF 就是用来修正这一点的：

- **TF**（词频）：某个词在这一篇文档里出现得多不多；
- **IDF**（逆文档频率）：这个词在**别的文档**里是不是到处都是。到处都有的词区分度低，IDF 就小。

`TF-IDF = TF × log(文档总数 / 包含该词的文档数)`

用文件夹里现成的 4 篇文本凑一个小语料库：

In [22]:
```
corpus_files = {
    '黄旭华传记序言': '科学家博物馆-黄旭华传记序言.txt',
    '三国演义': 'sanguo.txt',
    '人民日报-亚运会': '人民日报-杭州亚运会-科技智能.txt',
    '知了': '知了.txt',
}

corpus = {}
for name, f in corpus_files.items():
    raw = open(f, 'rb').read()
    for enc in ('utf-8', 'gb18030'):
        try:
            corpus[name] = raw.decode(enc).replace('\r\n', '\n')
            print(f'{name:<14s} {enc:<8s} {len(corpus[name]):>8d} 字符')
            break
        except UnicodeDecodeError:
            continue
```

```
黄旭华传记序言        utf-8       4030 字符
三国演义           gb18030    620098 字符
人民日报-亚运会       utf-8       2090 字符
知了             gb18030      1562 字符
```

In [23]:

```
import math

docs = {name: [w for w in jieba.cut(text) if is_meaningful(w)] for name, text in
corpus.items()}
doc_counts = {name: Counter(ws) for name, ws in docs.items()}

N = len(docs)
df_map = Counter()
for counts in doc_counts.values():
    df_map.update(counts.keys())        # 每个文档里出现一次算 1

def idf(term):
    return math.log((N + 1) / (df_map.get(term, 0) + 1)) + 1

target = '黄旭华传记序言'
counts = doc_counts[target]
total = sum(counts.values())
tfidf = {w: (c / total) * idf(w) for w, c in counts.items()}
ranked_tfidf = sorted(tfidf.items(), key=lambda x: x[1], reverse=True)[:20]

df_tfidf = pd.DataFrame(ranked_tfidf, columns=['词', 'TF-IDF'])
df_tfidf['词频'] = [counts[w] for w, _ in ranked_tfidf]
df_tfidf['出现在几篇文档'] = [df_map[w] for w, _ in ranked_tfidf]
df_tfidf.round(6)
```

Out[23]:

	词	TF-IDF	词频	出现在几篇文档
0	黄旭华	0.081456	59	1
1	核潜艇	0.044180	32	1
2	采集	0.040038	29	1
3	学术	0.030373	22	1
4	资料	0.028993	21	1
5	成长	0.020709	15	1
6	小组	0.019329	14	1
7	工作	0.018504	17	2
8	院士	0.017948	13	1
9	专业	0.017948	13	1
10	我国	0.016567	12	1
11	研制	0.016567	12	1
12	工程	0.015187	11	1
13	进行	0.014150	13	2
14	访谈	0.013806	10	1
15	技术	0.013062	12	2
16	第一代	0.011045	8	1
17	科学	0.011045	8	1
18	主要	0.011045	8	1
19	人生	0.009796	9	2

IDF 到底在起什么作用？直接看几个词的 IDF 值

In [24]:

```
probe = ['黄旭华', '核潜艇', '采集', '工作', '技术', '研究', '我国', '进行', '发展']
rows = []
for w in probe:
    rows.append({
        '词': w,
        '词频(TF分子)': counts.get(w, 0),
        '出现在几篇文档': df_map.get(w, 0),
        'IDF': round(idf(w), 4),
        'TF-IDF': round(tfidf.get(w, 0), 6),
    })
pd.DataFrame(rows)
```

Out[24]:

	词	词频(TF分子)	出现在几篇文档	IDF	TF-IDF
0	黄旭华	59	1	1.9163	0.081456
1	核潜艇	32	1	1.9163	0.044180
2	采集	29	1	1.9163	0.040038
3	工作	17	2	1.5108	0.018504
4	技术	12	2	1.5108	0.013062
5	研究	6	1	1.9163	0.008284
6	我国	12	1	1.9163	0.016567
7	进行	13	2	1.5108	0.014150
8	发展	2	2	1.5108	0.002177

这张表就是 TF-IDF 的全部秘密：`出现在几篇文档` 越多的词，`IDF` 越小。 `工作`、`技术`、`研究` 这类词在四篇文本里都出现过，IDF 被压到 1 附近； 而 `黄旭华`、`核潜艇` 只出现在这一篇里，IDF 最大，于是 TF-IDF 排到最前。

两种排名的对比

In [25]:

```
top_freq = [w for w, _ in articlelist[:15]]
top_tfidf = [w for w, _ in ranked_tfidf[:15]]

df_side = pd.DataFrame({'纯词频 Top15': top_freq, 'TF-IDF Top15': top_tfidf})
df_side.index = df_side.index + 1
df_side
```

Out[25]:

	纯词频 Top15	TF-IDF Top15
1	黄旭华	黄旭华
2	核潜艇	核潜艇
3	采集	采集
4	学术	学术
5	资料	资料
6	工作	成长
7	成长	小组
8	小组	工作
9	院士	院士
10	专业	专业
11	进行	我国
12	我国	研制
13	技术	工程
14	研制	进行
15	工程	访谈

In [26]:
```
# 分别看「词频高但 TF-IDF 掉出去」和「TF-IDF 升上来」的词
top_freq_set, top_tfidf_set = set(top_freq), set(top_tfidf)

print('词频高、但 TF-IDF 掉出前 15 的词（通用词，区分度低）：')
for w in sorted(top_freq_set - top_tfidf_set, key=lambda x: -counts[x]):
    print(f'  {w:<8s} 词频={counts[w]:<3d} 出现在 {df_map[w]}/{N} 篇  IDF=
{idf(w):.2f}')

print('\nTF-IDF 进前 15、但词频不是最高的词（更有辨识度）：')
for w in sorted(top_tfidf_set - top_freq_set, key=lambda x: -tfidf[x]):
    print(f'  {w:<8s} 词频={counts[w]:<3d} 出现在 {df_map[w]}/{N} 篇  IDF=
{idf(w):.2f}')
```

```
词频高、但 TF-IDF 掉出前 15 的词（通用词，区分度低）：
  技术       词频=12  出现在 2/4 篇  IDF=1.51

TF-IDF 进前 15、但词频不是最高的词（更有辨识度）：
  访谈       词频=10  出现在 1/4 篇  IDF=1.92
```

> **一句诚实话**：这里的语料只有 4 篇，IDF 只能取到少数几个值，所以 TF-IDF 排名和词频排名差别不大。 TF-IDF 真正的威力要在**成百上千篇文档**的语料里才显现——文档越多，「到处都有的词」被压得越狠。 这也解释了为什么 PPT 说「词频是重要指标，但不够深入」。

4 结论：从词频能读出什么

（1）谁被写得最多？

`黄旭华`（分词 53 次 / 原文匹配 59 次）、`核潜艇`（32 次）稳居前二，远超第三名。 这符合**人物传记序言**的特征：**传主 + 他的核心事业**，构成全文的绝对主线。

（2）这份文本「是什么类型」？

紧随其后的是 `采集`（29）、`学术`（22）、`资料`（21）、`小组`（14）、`访谈`（10）、`传记`（7）、`传主`（6）。 这些词不是传记正文的典型用词，而是**口述史 / 访谈类项目的作业语汇**—— 说明这段「序言」除了讲传主，还用了大量篇幅交代**资料是怎么采集来的。** 这是词频能读出的「文本体裁」信息，单看一两个词是看不出来的。

（3）齐普夫定律成立吗？

双对数图上点子落在一条向下的直线附近，**长尾形状成立**；拟合斜率约 −0.6 ~ −0.7， 负斜率确认，但因文本只有 4000 字，头部被截顶，达不到严格的 −1。

（4）为什么还需要 TF-IDF？

`工作`、`技术`、`研究` 这类词词频不低，但在哪篇文章里都常见，说明不了这篇文本的特点； `黄旭华`、`核潜艇` 次数略少，却一眼定位主题。TF-IDF 用「别的文档里有没有」来压低前者、抬高后者。

（5）这次作业最该记住的一件事：

> 词频统计不是「数一数」，而是**用一个可复现的数字回答一个关于文本的问题。** 数字本身不重要，**数字背后的对比**（过滤前后、两种计数口径、两种排名）才重要。
>
> 而且——**分词器会悄悄改变你的结论**：`黄旭华` 被「华为」抢走 4 次， 如果不加词典，你会少算 6 次，占比偏差 10%。停用词、词典、编码、换行符，每一个细节都会让「同一个文本」数出不同的结果。这就是文本分析里最容易被忽略的「可复现性问题」。

5 提交清单

内容	对应作业要求	本文位置
全文词频统计（分词→过滤→计数→排序→Top30）	要求 1 / 要求 2-1）	第 1 节
词云图	要求 1（对应 PPT 微词云）	1.5 节
指定词频统计（含“黄旭华”）	要求 1 / 要求 2-2）	第 2 节
指定词频柱状图	要求 1	2.4 节
分词口径诊断与人名修复	分析深度（加分）	2.3 节
齐普夫定律验证	拓展 2	3.1 节
TF-IDF 对比	拓展 1	3.2 节
分析与结论	全篇	第 4 节